

Local-First Chain Notes: Capturing AI Conversations as Searchable Records

A practice report from building a desktop toolkit

Ada Osei · Independent Research Tunde Bakare · Local Tools Lab

Workshop on Local-First Software · 2026

Abstract

AI assistants produce long, valuable conversations that usually evaporate when the chat window closes. This paper describes Chain Notes, a local-first workspace that stores external AI exchanges as question-and-answer records on the user's own machine. Notes persist in a versioned JSON store, search tolerates typos through fuzzy matching [1], and each exchange exports to PDF or print. The design follows local-first principles [2]: the application owns its data, works offline, and treats synchronization as a future concern rather than a precondition.

Keywords: local-first software, note-taking, fuzzy search, desktop applications

1 Introduction

Conversations with AI systems have become working documents: debugging sessions, design reviews, and literature surveys now happen inside chat transcripts. Yet most assistants keep those transcripts behind a network boundary, searchable only by exact scrollbar. When the session ends, the knowledge is effectively lost.

Chain Notes takes the opposite stance. Each exchange is saved locally the moment it is captured, as a small structured record with three fields:

- **Question** — what was asked, verbatim.
- **Answer** — what was generated, verbatim.
- **Title** — a short human label for scanning.

Three fields turn out to be enough. Titles make lists skimmable, questions preserve intent, and answers preserve evidence. Everything else — tags, timestamps, exports — is derived.

2 Related Work

Personal knowledge management has converged on two poles: plain-text systems that compose well with other tools [3], and database-backed applications with rich queries [4]. Chain Notes borrows the plain-text instinct — records are human-readable JSON — while keeping the query power of a database for search and export.

| The best note is the one you can find again six months later.

Local-first software [2] argues that applications should own their data and work offline. Chain Notes applies that argument to AI transcripts specifically: because the content already exists as text, no server round-trip is ever required to read, search, or export it.

3 Design

3.1 Storage

Notes live in a single versioned JSON document (`{ "version": 1, "notes": [...] }`). Writes are atomic: the backend serializes to a temporary file and renames it, so readers never observe a partial state. The schema version leaves room for migrations without breaking existing installs.

The storage invariant can be summarized as $\$writes \rightarrow durable\ state\$$ when the shutdown flush completes.

3.2 Search

Exact substring search fails on typos, which are common when recalling half-remembered prompts. The workspace therefore benchmarks several fuzzy engines and ships the fastest selective one [1]. The benchmark harness stays in the repository so the choice can be revisited with evidence rather than opinion.

3.3 Export

Each exchange renders to PDF through one of three engines, or to the system print dialog for a true Save-as-PDF flow. Markdown in questions and answers — code fences, lists, quotes — survives export with monospace blocks and preserved indentation:

4 Implementation Notes

The desktop shell pairs a Julia backend with a Preact frontend over a small typed RPC bridge, as sketched below:

```
flowchart LR
  UI[Preact paper UI] --> RPC[Typed RPC bridge]
  RPC --> API[Julia Backend]
  API --> PDF[Documents/*.pdf]
```

The system architecture is also shown in Figure 1:

Figure 1

Vector preview lives in the reader.
Upload PNG or JPEG to embed raster art.

Native shell data flow

PDF bytes are generated in the frontend, encoded once, and handed to a `savePdf` binding that validates the filename and writes atomically into the user's Documents folder. A typical save round-trips in well under a second:

```
const bytes = await generateChainPdfBytes("jspdf", entries);
const saved = await backend.savePdf(filename, pdfBytesToBase64(bytes));
// saved.path -> "Documents/chain-notes-2026-09-04.pdf"
```

The same block model drives screen reading, print CSS, and all three PDF engines, so a layout fix lands everywhere at once [5].

5 Conclusion

Saving AI conversations locally, as structured question-and-answer records, converts ephemeral chats into a durable personal corpus. Fuzzy search makes the corpus forgiving; PDF and print export make it portable. Future work includes cross-device sync with conflict-free merges and spaced-repetition review drawn from saved exchanges.

References

1. [1] fuzzysort: fast SublimeText-like fuzzy search for JavaScript. MIT License.
2. [2] M. Kleppmann et al. Local-first software: you own your data, in spite of the cloud. Onward! 2019.
3. [3] Obsidian: a second brain, on top of local Markdown files.
4. [4] Notion: connected workspace for docs, wikis, and projects.
5. [5] webview: a tiny cross-platform library for building modern desktop apps with web UIs.